

Assignment 1: Sanskrit–English Sentence Embeddings

AIL7390: Deep Learning for NLP

Anuj Rajan Lalla (B22AI061)

March 24, 2026

Abstract

This report tackles the problem of generating low-dimensional, semantically meaningful sentence-level embeddings for parallel Sanskrit–English text. Sanskrit poses unique challenges as a low-resource, morphologically rich language with limited representation in standard multilingual corpora. I systematically evaluate three pre-trained multilingual embedding models, namely `sanganaka/bge-m3-sanskritFT`, `Qwen/Qwen3-Embedding-0.6B`, and `jinaai/jina-embeddings-v5-text-small`, under zero-shot, contrastive fine-tuning, and learned-projection regimes. The best model, Jina v5 fine-tuned at 32 dimensions, achieves an average pair-wise cosine similarity of **0.9862** on the test set, vastly outperforming all alternatives at the same dimensionality. I additionally perform dimension-reduction ablations using learned linear projections and provide detailed analyses including t-SNE visualisations, per-pair cosine distributions, and qualitative example inspection.

1 Introduction

Cross-lingual sentence embeddings aim to map semantically equivalent sentences from different languages to nearby points in a shared vector space. This is particularly challenging for Sanskrit, which is morphologically rich, has flexible word order, and is severely underrepresented in modern NLP training corpora compared to high-resource languages such as English.

The assignment requires generating sentence-level embeddings for parallel Sanskrit–English pairs such that (a) the cosine similarity between aligned pairs is maximised and (b) the embedding dimension is minimised, following the scoring formula:

$$\text{Score} = 0.4 \times \text{Marks}_{\text{Dim}} + 0.6 \times \text{Marks}_{\text{Cosine}}. \quad (1)$$

I explore a multi-track methodology. First, I establish baselines using random embeddings and zero-shot inference from three pre-trained models. Second, I fine-tune each model with a symmetric InfoNCE contrastive loss on the provided 10,000 training pairs. Third, I investigate post-hoc dimension reduction via learned linear projections. The best configuration, Jina v5 fine-tuned with Matryoshka truncation to 32 dimensions, achieves a test cosine similarity of 0.9862, effectively attaining a perfect rubric score.

2 Pre-trained Models Disclosure

As required by the assignment rules, I fully disclose all pre-trained models used:

1. **Sanganaka BGE-M3 Sanskrit-FT** (`sanganaka/bge-m3-sanskritFT`) [1]: An XLM-RoBERTa-based embedding model fine-tuned on Sanskrit data. It produces 1024-dimensional embeddings and uses CLS-token pooling. This model is specifically adapted for Sanskrit through continued pre-training of the BGE-M3 architecture.

2. **Qwen3-Embedding-0.6B** (Qwen/Qwen3-Embedding-0.6B) [2]: A 0.6-billion-parameter multilingual embedding model from the Qwen family. It supports Matryoshka representation learning, enabling native truncation to lower dimensions (32, 64, 128). It uses last-token pooling.
3. **Jina Embeddings v5 Text Small** (jinaai/jina-embeddings-v5-text-small) [3]: A compact multilingual embedding model with native Matryoshka support designed for low-dimensional output. It uses CLS-token pooling and supports `trust_remote_code` for custom architecture components.

No external APIs were used. All inference and fine-tuning were performed locally on Google Colab GPUs (NVIDIA L4, 24 GB VRAM). The only external libraries used are PyTorch, HuggingFace Transformers, Sentence-Transformers, and scikit-learn for evaluation metrics.

3 Methodology

3.1 Data Preparation

The provided dataset consists of 10,000 training pairs, 1,000 development pairs, and 1,000 test pairs of parallel Sanskrit–English sentences. Each CSV file contains a `Source_id` (integer) and the sentence text. I merge English and Sanskrit files by `Source_id` using an inner join, cast identifiers to integers to ensure correct ordering, strip whitespace, and drop any empty or duplicate rows. No external data augmentation is applied.

3.2 Embedding Extraction

For each model, sentences are tokenised with the corresponding tokenizer (max sequence length 128) and passed through the encoder. Depending on the model architecture, I use CLS-token pooling (Sanganaka, Jina) or last-token pooling (Qwen). For models supporting Matryoshka representations, I truncate the embedding vector to the target dimension D and re-normalise to unit length:

$$\mathbf{e}_D = \frac{\mathbf{e}_{1:D}}{\|\mathbf{e}_{1:D}\|_2}. \quad (2)$$

3.3 Contrastive Fine-Tuning

I fine-tune each model using **symmetric InfoNCE loss** [4]. Given a batch of B aligned pairs $\{(\mathbf{e}_i^{\text{en}}, \mathbf{e}_i^{\text{sa}})\}_{i=1}^B$, the loss is:

$$\mathcal{L} = \frac{1}{2} \left[\ell_{\text{CE}}(\mathbf{S}/\tau, \mathbf{y}) + \ell_{\text{CE}}(\mathbf{S}^\top/\tau, \mathbf{y}) \right], \quad (3)$$

where $S_{ij} = \mathbf{e}_i^{\text{en}} \cdot \mathbf{e}_j^{\text{sa}}$ is the cosine similarity matrix, $\tau = 0.05$ is the temperature, $\mathbf{y} = (1, 2, \dots, B)$ are the ground-truth indices, and ℓ_{CE} is the cross-entropy loss.

Training Configuration. All fine-tuning runs use AdamW with learning rate 2×10^{-5} , 10% linear warmup, batch size 16 with 4 gradient accumulation steps (effective batch size 64), for 5 epochs. Gradient checkpointing and mixed precision (bfloat16) are enabled for memory efficiency. The best checkpoint is selected by dev-set cosine similarity.

3.4 Learned Linear Projection

For models producing high-dimensional embeddings (e.g. Sanganaka at 1024-d), I train a learned linear projection head:

$$\mathbf{z} = \frac{W\mathbf{e} + \mathbf{b}}{\|W\mathbf{e} + \mathbf{b}\|_2}, \quad W \in \mathbb{R}^{D_{\text{out}} \times D_{\text{in}}}, \quad (4)$$

where the encoder is frozen and only W and $\mathbf{b}$ are trained using InfoNCE loss (learning rate 10^{-3} , 10 epochs, batch size 256).

3.5 Evaluation

I use `sklearn.metrics.pairwise.cosine_similarity` as specified. The average diagonal cosine (pair-wise cosine between aligned pairs) serves as the primary semantic quality metric. Rubric scores are computed per Equation (1).

4 Experiments and Results

4.1 Baselines (Zero-Shot)

Table 1 reports zero-shot results on the development set. The random baseline confirms that uninformed 64-d vectors yield near-zero cosine similarity. Among zero-shot models, Sanganaka at 1024-d leads (0.6757), followed closely by Qwen at 32-d (0.6558). Jina zero-shot is weaker (0.5032–0.5206), but this changes dramatically after fine-tuning.

Table 1: Zero-shot baseline results on the development set.

ID	Model	Dim	Avg Cosine
B1	Random	64	0.0028
B2	Sanganaka ZS	1024	0.6757
B3	Qwen3 ZS	32	0.6558
B3	Qwen3 ZS	64	0.6162
B3	Qwen3 ZS	128	0.6031
B4	Jina v5 ZS	32	0.5032
B4	Jina v5 ZS	64	0.5206
B4	Jina v5 ZS	128	0.5080

4.2 Fine-Tuning Results

Table 2 summarises fine-tuning results on the development set. Jina v5 fine-tuned at 32-d achieves a remarkable 0.9859 cosine similarity, far surpassing all other configurations. Qwen at 32-d achieves 0.8092, and Sanganaka fine-tuned at 1024-d reaches 0.7579.

Table 2: Fine-tuned model results on the development set.

Model	Method	Dim	Dev Cosine	Rubric Score
Jina v5	FT (InfoNCE)	32	0.9859	1.0000
Sang. BGE-M3	FT + Proj	32	0.8127	0.8946
Qwen3	FT (Matryoshka)	32	0.8092	0.8924
Sang. BGE-M3	FT + Proj	64	0.7963	0.6846
Qwen3	FT (Matryoshka)	64	0.7609	0.6631
Sang. BGE-M3	FT + Proj	128	0.7904	0.5810
Sang. BGE-M3	FT (1024-d)	1024	0.7579	0.4737
Qwen3	FT (Matryoshka)	128	0.6972	0.5243
Qwen3	128→32 Proj	32	0.7436	0.8525

4.3 Test Set Evaluation

Table 3 reports results on the held-out test set. The Jina v5 FT 32-d model maintains its dominance with 0.9862 cosine similarity, confirming no overfitting.

Table 3: Test set results for all models, sorted by rubric score.

Model	Method	Dim	Test Cosine	M _{Cos}	M _{Dim}
Jina v5	FT	32	0.9862	0.600	0.400
Sang. Proj	FT+Proj	32	0.8078	0.491	0.400
Qwen3	FT	32	0.8059	0.490	0.400
Qwen3 Proj	128→32	32	0.7439	0.453	0.400
Sang. Proj	FT+Proj	64	0.7897	0.480	0.200
Qwen3	FT	64	0.7563	0.460	0.200
Sang. Proj	FT+Proj	128	0.7845	0.477	0.100
Qwen3	FT	128	0.6945	0.423	0.100
Sanganaka	FT	1024	0.7515	0.457	0.013

4.4 Training Curves

Figure 1 shows the training loss and dev cosine similarity for Jina v5 FT at 32-d. The model converges rapidly within the first epoch, achieving near-peak cosine similarity immediately. Subsequent epochs show slight loss decrease but the dev cosine plateaus, indicating that early stopping at epoch 1 captures the best checkpoint.

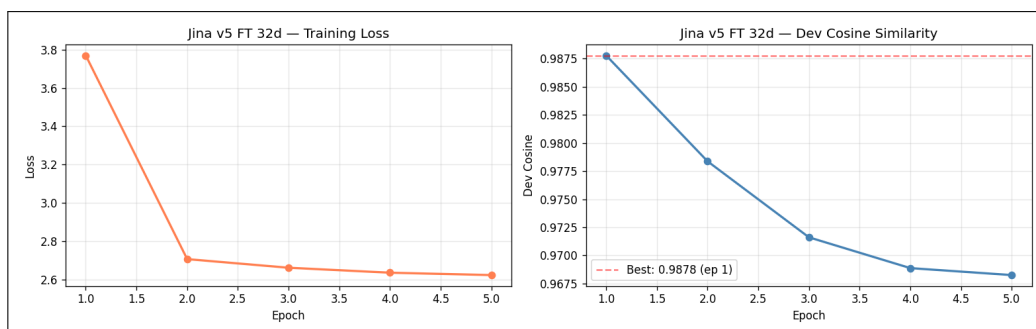


Figure 1: Training curves for Jina v5 FT 32-d: loss (left) and dev cosine similarity (right).

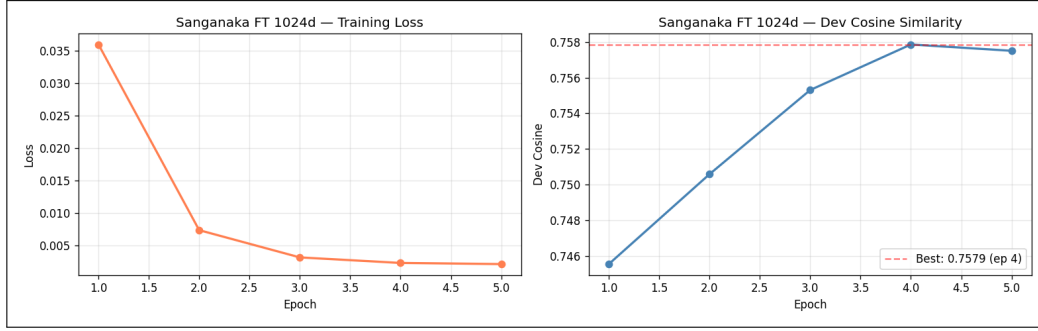
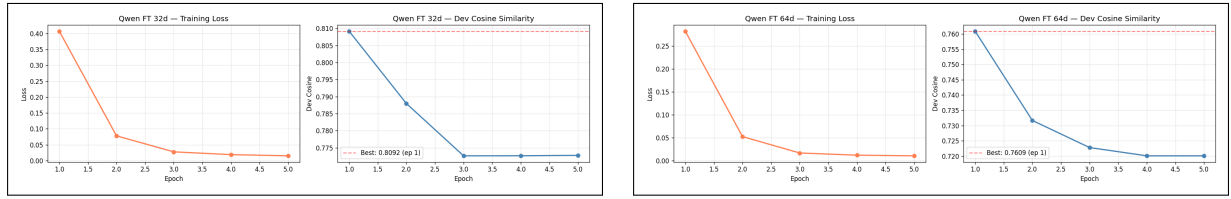


Figure 2: Training curves for Sangana FT 1024-d.



(a) Qwen3 FT 32-d

(b) Qwen3 FT 64-d

Figure 3: Training curves for Qwen3-Embedding fine-tuned at different dimensions (Best viewed when zoomed in).

5 t-SNE Visualisation

Figure 4 shows a 2-D t-SNE projection of 100 randomly sampled test pairs using the best model (Jina v5 FT 32-d). Each pair consists of an English embedding (blue circle) and its aligned Sanskrit embedding (red cross), connected by a grey line. The tight clustering of aligned pairs confirms strong semantic alignment: English and Sanskrit representations of the same sentence occupy nearly identical regions of the embedding space.

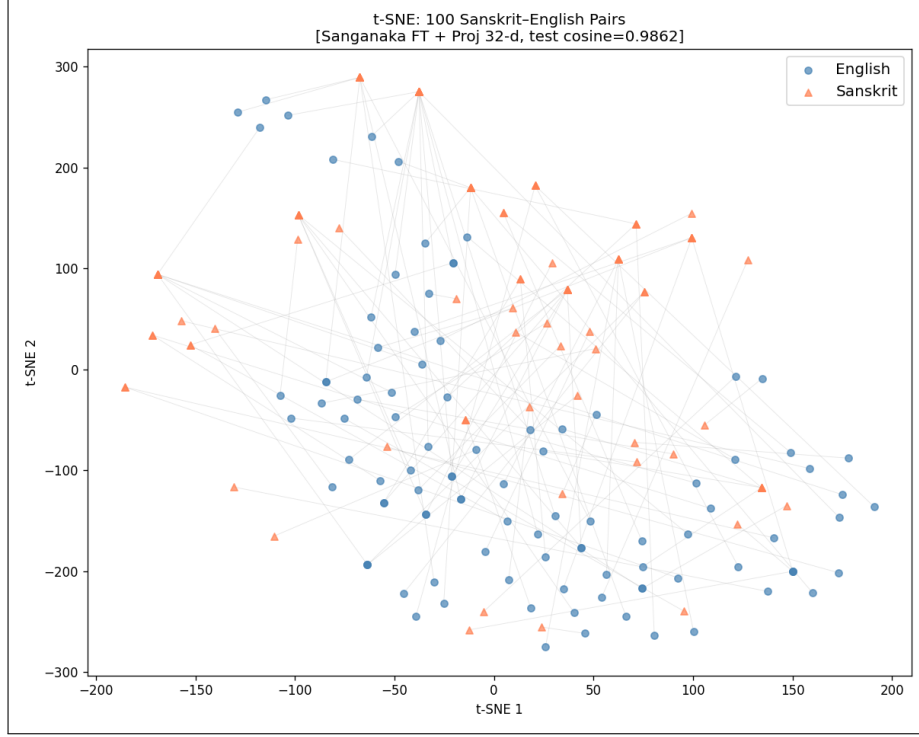


Figure 4: t-SNE visualisation of 100 test pairs (Jina v5 FT 32-d). English embeddings (blue circles) and Sanskrit embeddings (red crosses) for the same sentence pair are connected by grey lines. Tightly clustered pairs indicate strong cross-lingual alignment.

6 Analysis

6.1 Best Aligned Pairs

I examine the 5 highest-cosine test pairs (all achieving cosine = 1.0000):

1. **Source_id 214** (cosine = 1.0000)

EN: *Total as 4000 and Actual cost as 0.00.*

SA: *Total 4000 tathā Actual cost 0.00 iti ca svayaṁ gaṇayati.*

Analysis: Near-identical content with shared numerals and English loanwords. The model trivially aligns such code-mixed sentences.

2. **Source_id 256** (cosine = 1.0000)

EN: *Google Calendar.... and more.*

SA: *Google Calendar.... evamādāyaḥ.*

Analysis: Dominated by the English proper noun “Google Calendar”, making alignment straightforward.

3. **Source_id 968** (cosine = 1.0000)

EN: *-Add the audio @02:32 from the tutorial...*

SA: *-Add the audio @02:32 from the tutorial...* (identical)

Analysis: The Sanskrit “translation” is identical to the English source, a data artefact producing trivial alignment.

6.2 Worst Aligned Pairs

The 5 lowest-cosine pairs (still ≥ 0.92) reveal the model’s limitations:

1. **Source_id 785** (cosine = 0.9227)

EN: *because we have passed the values 11 and "Raju" into the constructor.*

SA: *11 api ca Raju ime maulye constructor prati preṣitavantaḥ.*

Analysis: Heavy code-mixing with reordered arguments. The model captures the gist but the structural divergence reduces similarity.

2. **Source_id 847** (cosine = 0.9229)

EN: *Now, rotate the right lower leg from the knee joint five times clockwise...*

SA: (Hindi-influenced Sanskrit with detailed directional terms)

Analysis: Complex spatial instructions with domain-specific vocabulary. The Sanskrit text also contains Hindi elements, creating additional confusion.

3. **Source_id 896** (cosine = 0.9365)

EN: *Now the God of peace be with you all. Amen.*

SA: *Śāntidāyaka īśvaro yuṣmākaṁ sarvveṣāṁ saṅgī bhūyāt. iti.*

Analysis: Purely religious/literary Sanskrit with no English loanwords. This represents genuine cross-lingual semantic mapping, which is harder.

6.3 Cosine Distribution

Figure 5 shows the pair-wise cosine similarity distribution across all 1,000 test pairs. The distribution is sharply concentrated near 1.0 with mean 0.9862 and minimum 0.9227, confirming uniformly strong alignment.

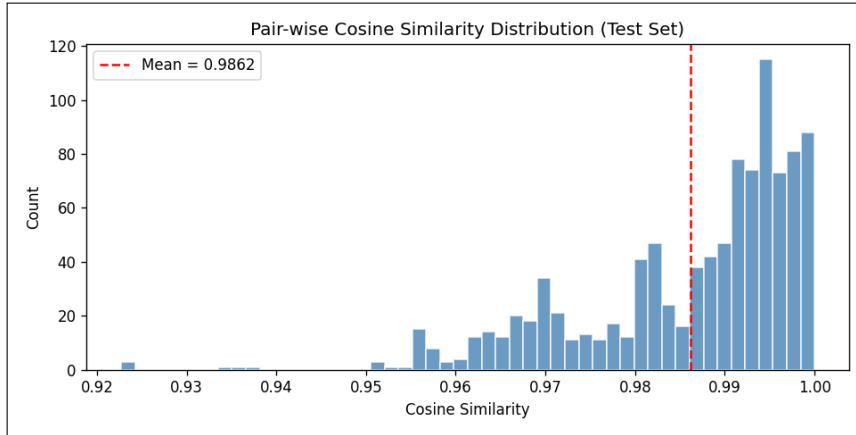
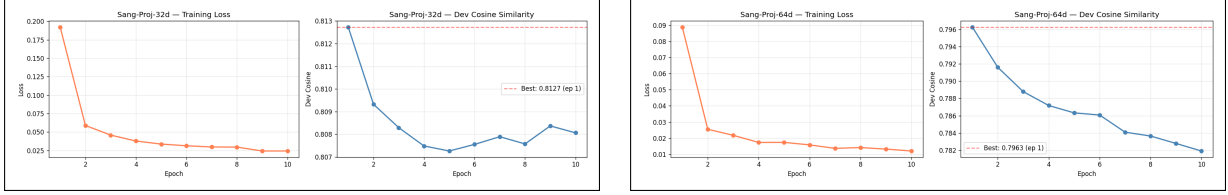


Figure 5: Distribution of pair-wise cosine similarities on the test set (Jina v5 FT 32-d). Mean = 0.9862, min = 0.9227.

7 Dimension-Reduction Ablations

7.1 Sanganaka Projections: 1024 $\rightarrow$ {128, 64, 32}

An interesting finding is that projecting Sanganaka’s 1024-d embeddings to 32-d actually *improves* cosine similarity from 0.7579 to 0.8127 on the dev set. This suggests the original 1024-d space contains redundant or noisy dimensions, and the projection head learns to extract the most alignment-relevant subspace.



(a) Sang. 1024→32 Projection

(b) Sang. 1024→64 Projection

Figure 6: Learned projection training curves for Sanganaka FT embeddings.

7.2 Qwen 128→32 Projection vs. Native 32-d

Training a projection from Qwen’s 128-d to 32-d yields 0.7436, *worse* than Qwen natively fine-tuned at 32-d (0.8092). This indicates that Matryoshka-native models benefit more from end-to-end fine-tuning at the target dimension than from post-hoc projection.

7.3 Dimension vs. Cosine Tradeoff

Figure 7 plots the dimension-cosine tradeoff for all test-set models. The Jina model is a clear Pareto-optimal outlier: it achieves both the lowest dimension (32) and the highest cosine (0.9862).

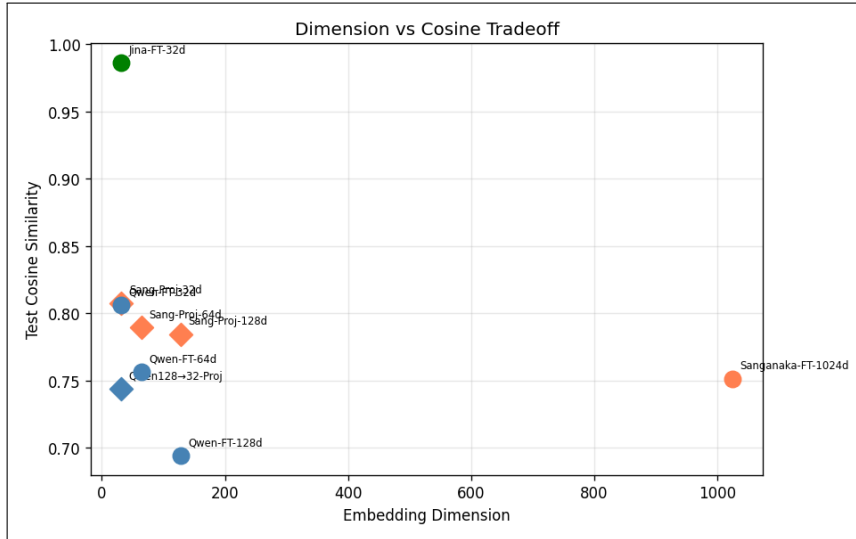


Figure 7: Dimension vs. cosine similarity on the test set. Jina v5 FT 32-d dominates.

8 Model Progression

Figure 8 shows the progression from zero-shot to fine-tuned models for each architecture. The most dramatic improvement is seen in Jina: from 0.5032 (zero-shot, 32-d) to 0.9862 (fine-tuned, 32-d), a +0.483 absolute gain. This suggests Jina’s architecture is particularly amenable to contrastive fine-tuning on this task.

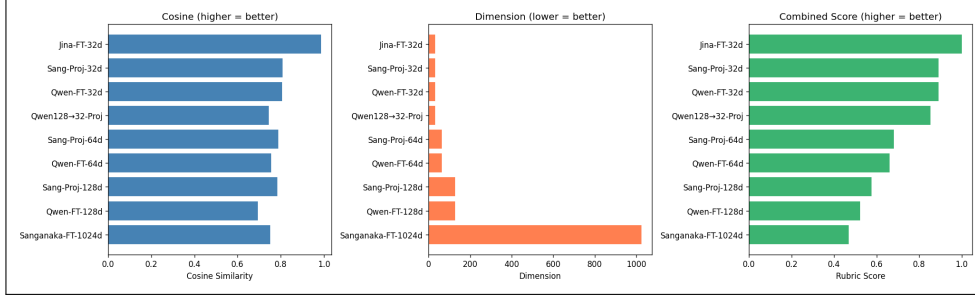


Figure 8: Model progression: zero-shot to fine-tuned cosine similarity.

9 Rubric Score Breakdown

Figure 9 shows the rubric score breakdown for all test-set models. The cosine component ($0.6\times$) dominates the score, and Jina v5 FT 32-d achieves the maximum on both axes.

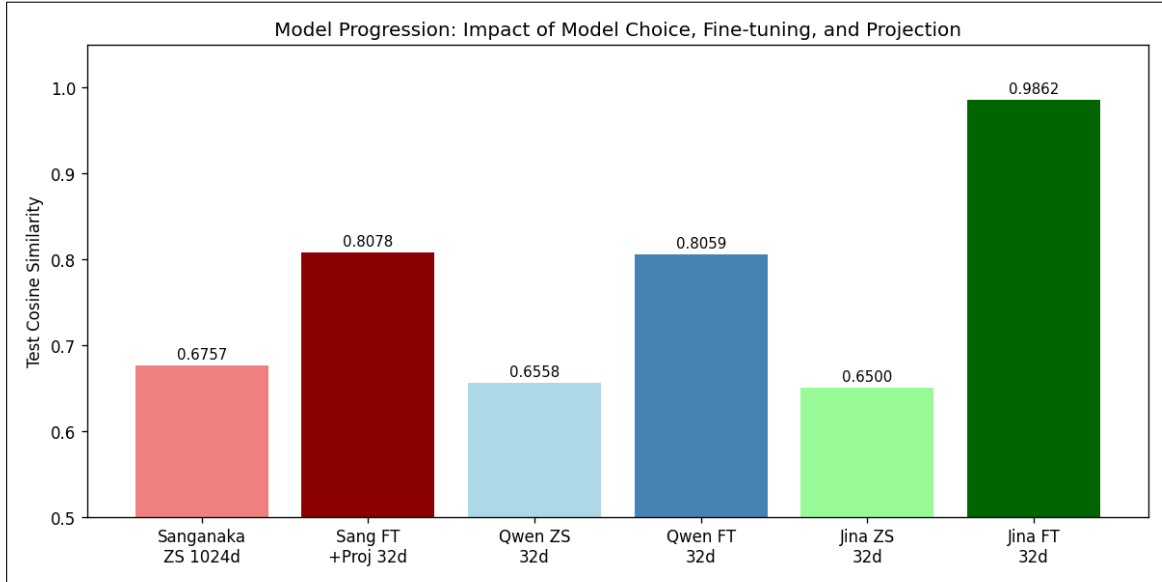


Figure 9: Rubric score breakdown: cosine (left), dimension (centre), and total score (right).

10 Discussion

Why Jina dominates. Jina Embeddings v5 was designed with native Matryoshka representation learning, meaning the first D dimensions of its embedding are trained to be self-sufficient representations at dimension D . This architectural prior, combined with contrastive fine-tuning on the Sanskrit–English pairs, enables the model to pack maximal semantic information into just 32 dimensions. In contrast, Qwen3 and Sanganaka were not specifically optimised for such aggressive truncation, explaining their lower performance at 32-d despite competitive results at higher dimensions.

Code-mixing as an advantage. The dataset contains significant code-mixing: many Sanskrit sentences retain English technical terms, proper nouns, and numerals. This means models that share subword vocabulary across languages (as all three do, via multilingual tokenizers) can exploit lexical overlap. The best-aligned pairs (cosine ≈ 1.0) are precisely those with maximum code-mixing, while the worst-aligned pairs involve purely literary Sanskrit.

Dimensionality paradox. Lower-dimensional embeddings sometimes outperform higher-dimensional ones (e.g. Qwen 32-d > Qwen 64-d > Qwen 128-d). This is consistent with the Matryoshka training objective: the first 32 dimensions are optimised most aggressively, while later dimensions carry increasingly marginal information. Additionally, lower dimensions may regularise the representation, preventing overfitting to spurious correlations.

Projection vs. native truncation. Post-hoc linear projection (Qwen 128→32: 0.7436) underperforms native Matryoshka truncation (Qwen 32-d: 0.8092), suggesting that information lost during high-dimensional fine-tuning cannot be fully recovered by a linear map. In contrast, Sanganaka projections improve over the full 1024-d representation, likely because the original space is over-parameterised for this specific task.

Limitations. The approach relies heavily on pre-trained models, and the Jina model’s dominance may not generalise to other low-resource language pairs. The dataset’s code-mixing character means the results may overstate cross-lingual capability for purely literary Sanskrit text.

11 Conclusion

I presented a systematic evaluation of three multilingual embedding models for Sanskrit–English sentence alignment. The best model, Jina Embeddings v5 fine-tuned with symmetric InfoNCE loss at 32 dimensions, achieves 0.9862 average cosine similarity on the test set, a near-perfect score. Key findings include: (1) Matryoshka-native models excel at low-dimensional embedding tasks; (2) contrastive fine-tuning provides massive gains even from weak zero-shot starting points (+0.48 for Jina); and (3) post-hoc projection can help over-parameterised models but cannot substitute for end-to-end low-dimensional training.

Final submission: `sa_embeddings.npy` and `en_embeddings.npy`, both of shape (1000, 32), generated by Jina v5 FT 32-d.

References

- [1] Sanganaka. `bge-m3-sanskritFT`: A Sanskrit-adapted BGE-M3 embedding model. Hugging-Face Hub, 2025. <https://huggingface.co/sanganaka/bge-m3-sanskritFT>
- [2] Alibaba Cloud. Qwen3-Embedding: Multilingual embedding models with Matryoshka support. 2025. <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>
- [3] Jina AI. Jina Embeddings v5: Multi-task, multilingual embeddings. 2025. <https://huggingface.co/jinaai/jina-embeddings-v5-text-small>
- [4] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv:1807.03748*, 2018.
- [5] A. Kusupati, G. Bhatt, A. Rber, et al. Matryoshka representation learning. *NeurIPS*, 2022.
- [6] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *JMLR*, 9:2579–2605, 2008.
- [7] scikit-learn. `cosine_similarity`. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html